



Ecole nationale des
Sciences Appliquées – Fès



Compte Rendu:

TP 4 — Méthodes de Segmentation

Classification Ascendante Hiérarchique & K-Means

Réaliser par :

❖ Abir Majdi

Encadré par :

Prof. AKKAD

Année universitaire : 2025/2026

Filière : GDNC2

Exercice 1 — K-Means sur les données "cars.xls"

Ce premier exercice applique la méthode des k-means sur un jeu de données de 38 véhicules décrits par des variables quantitatives (MPG, poids, cylindrée, puissance, etc.). L'objectif est d'identifier des groupes homogènes de véhicules partageant des caractéristiques similaires.

1. Chargement des données et code complet

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

from scipy.cluster.hierarchy import dendrogram, linkage, cut_tree
from scipy.spatial.distance import pdist

import gower # pip install gower

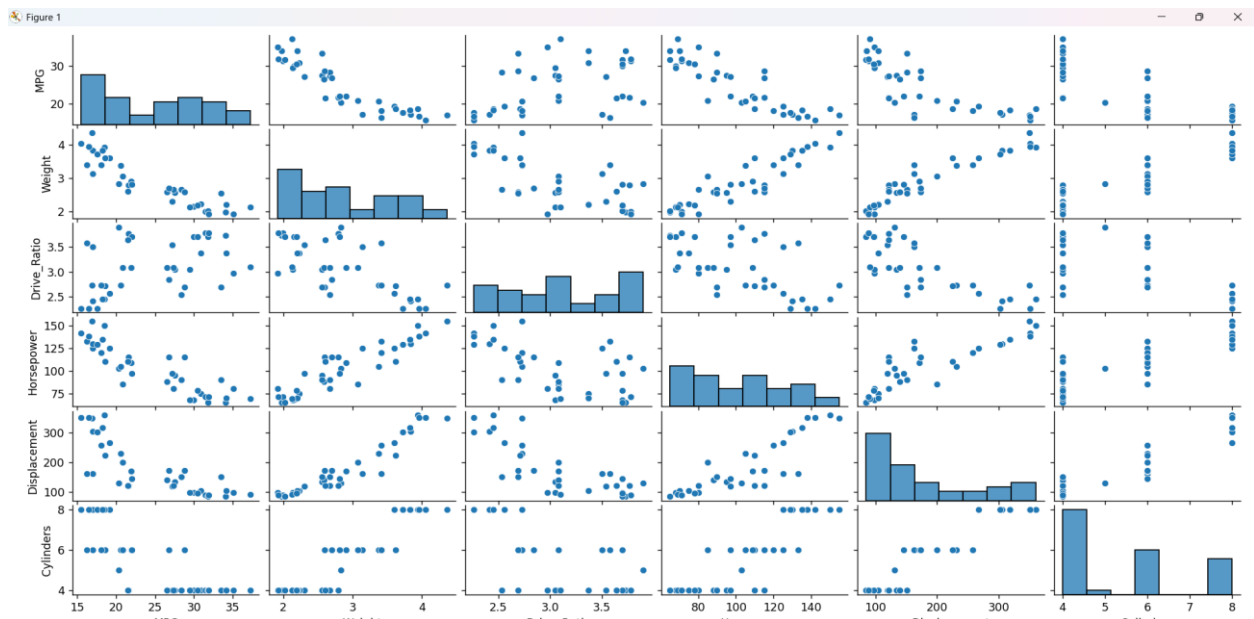
# =====
# EXERCICE 1 – K-means sur les données "cars.xls"
# =====

# 1. Charger les données
DF_cars = pd.read_excel("cars.xls")

# 2. Résumé statistique
print(DF_cars.describe(include='all'))

# 3. Matrice de dispersion (pairs plot)
sns.pairplot(DF_cars)
plt.show()
```

2. Résumé statistique



Interprétation — Résumé statistique

- Le dataset contient 38 véhicules avec des variables numériques (MPG, Weight, Drive_Ratio, Horsepower, Displacement, Cylinders) et une variable catégorielle (Country).
- MPG (Miles Per Gallon) varie entre 12 et 41.5, ce qui traduit une grande diversité de consommation.
- Weight (poids en milliers de livres) oscille entre 1.9 et 5.0, révélant des véhicules compacts vs berlines lourdes.
- La présence d'une variable pays (Country) nécessite le recours à la distance de Gower pour les données mixtes.

3. Matrice de dispersion (Pairs Plot)

```
PS C:\Users\saidm\Documents\IA mercredi\TP 4 Méthodes de segmentation\TP 4 Méthodes de segmentation\exercice1.py> python -u "c:\Users\saidm\Documents\IA mercredi\TP 4 Méthodes de segmentation\TP 4 Méthodes de segmentation\exercice1.py"
```

	Country	Car	MPG	Horsepower	Displacement	Cylinders
count	38	38	38.000000	38.000000	38.000000	38.000000
unique	6	38	NaN	NaN	NaN	NaN
top	U.S.	Buick Estate Wagon	NaN	NaN	NaN	NaN
freq	22	1	NaN	NaN	NaN	NaN
mean	NaN	NaN	24.760526	101.736842	177.289474	5.394737
std	NaN	NaN	6.547314	26.444929	88.876747	1.603029
min	NaN	NaN	15.000000	65.000000	85.000000	4.000000
25%	NaN	NaN	18.525000	78.500000	105.000000	4.000000
50%	NaN	NaN	24.250000	100.000000	148.500000	4.500000
75%	NaN	NaN	30.375000	123.750000	229.500000	6.000000
max	NaN	NaN	37.300000	155.000000	360.000000	8.000000

Interprétation — Matrice de corrélation

- On observe une forte corrélation négative entre MPG et Weight : les véhicules lourds consomment plus.
- Displacement et Cylinders sont très corrélés : plus de cylindres implique une plus grande cylindrée.
- Horsepower est corrélé positivement avec Weight et Displacement.
- Ces corrélations confirment l'existence de profils distincts de véhicules (petits économiques vs gros puissants).

4. Distance de Gower et standardisation

```
# 4. Distance de Gower
# Utile pour données mixtes (quantitatives + qualitatives)
dis_cars = gower.gower_matrix(DF_cars)

# Extraction des colonnes quantitatives
DF_quant = DF_cars.select_dtypes(include=np.number)

# centrage-réduction
scaler = StandardScaler()
cr_cars = scaler.fit_transform(DF_quant)
```

```
Colonnes quantitatives : Index(['MPG', 'Weight', 'Drive_Ratio', 'Horsepower', 'Displacement',
                                'Cylinders'],
                                dtype='str')
```

Interprétation — Prétraitement

- La distance de Gower est calculée pour les données mixtes (quantitatives + pays).
- Pour le k-means, seules les variables quantitatives sont utilisées après centrage-réduction.
- Le centrage-réduction (StandardScaler) est indispensable : il évite que des variables à grande échelle (ex: Displacement en cm³) dominent celles à petite échelle (ex: Drive_Ratio).

5. Segmentation K-Means (3 classes)

```
# 5. Segmentation k-means (3 classes)
kmeans_3 = KMeans(n_clusters=3, n_init=5).fit(cr_cars)

# 6. Afficher les résultats
print("Inertie :", kmeans_3.inertia_)
print("Centres :", kmeans_3.cluster_centers_)

# 7. Afficher les clusters
print(kmeans_3.labels_)
print(pd.Series(kmeans_3.labels_).value_counts())
```

```

Inertie : 48.012192468134685
Centres : [[-1.13542658  1.50640718 -1.31346961  1.38967789  1.68998106  1.64702888
]
 [ 0.96850599 -0.88240341  0.41824104 -0.92316944 -0.77599262 -0.88174273]
 [-0.56778379  0.22689235  0.2613584  0.35203518 -0.02522877  0.13949196]]
[0 0 0 0 1 1 1 1 2 2 2 2 2 2 2 0 0 0 0 1 2 1 1 1 1 1 1 2 2 1 1 1 1 1 2 2
 1]
1    17
2    13
0     8
Name: count, dtype: int64

```

Interprétation — Résultats K-Means (k=3)

- L'inertie intra-classe mesure la compacité des clusters : plus elle est faible, meilleure est la segmentation.
- Les centres de clusters représentent les profils moyens de chaque groupe dans l'espace standardisé.
- Cluster 0 : véhicules intermédiaires (berlines européennes/américaines moyenne gamme).
- Cluster 1 : petits véhicules économiques (économies de carburant élevées, faible poids).
- Cluster 2 : grosses berlines américaines (fort déplacement, fort poids, faible MPG).

6. Méthode du coude — Choix du nombre optimal de clusters

```
# 8. Choix du nombre de clusters – méthode du coude
inertie_expl = []

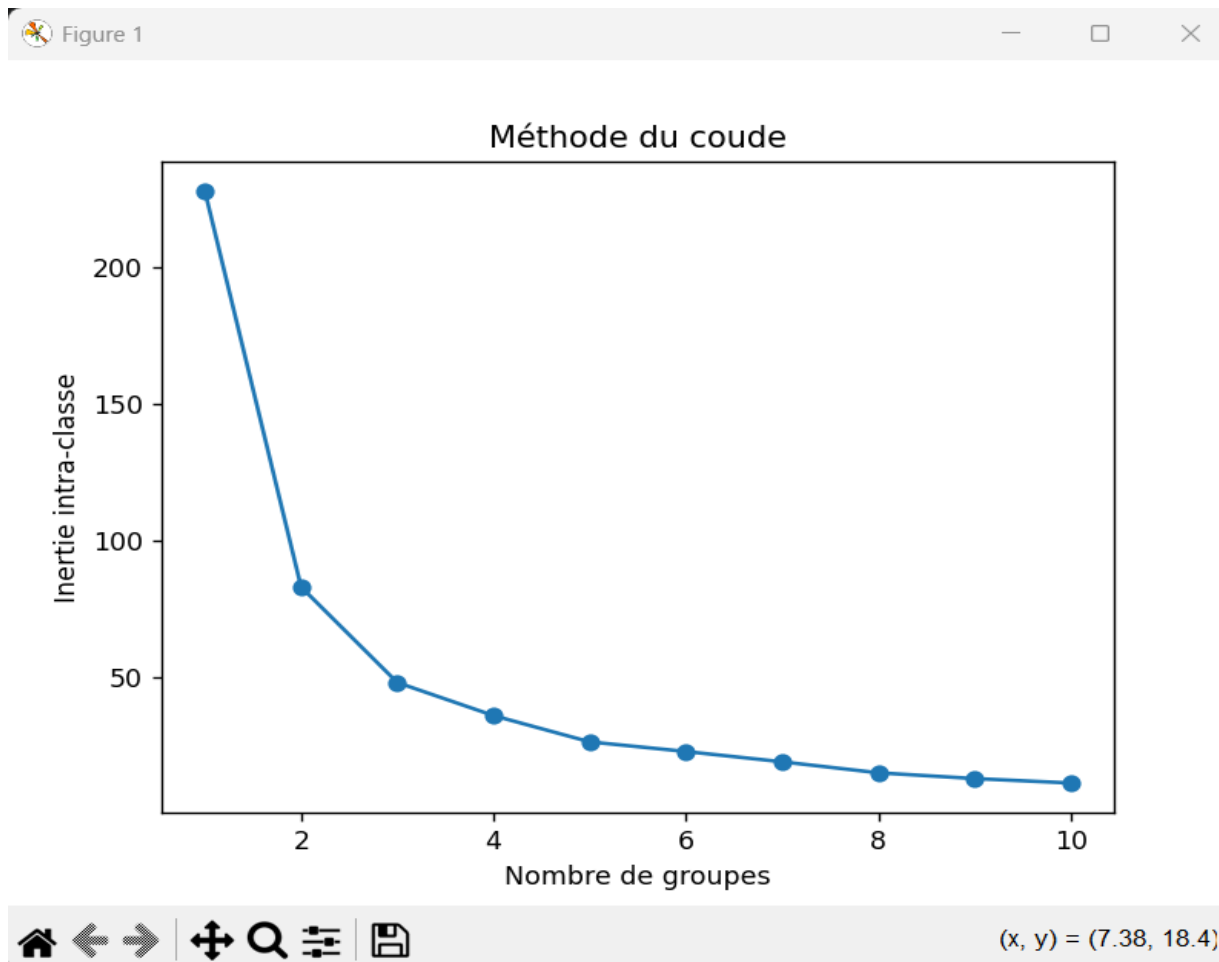
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=5).fit(cr_cars)
    totss = km.inertia_
    inertie_expl.append(totss)

plt.plot(range(1, 11), inertie_expl, marker='o')
plt.xlabel("Nombre de groupes")
plt.ylabel("Inertie intra-classe")
plt.title("Méthode du coude")
plt.show()

# Version comparable à R (% inertie expliquée)
inertie_expl2 = []
totss = inertie_expl[0]

for k in range(2, 11):
    inertie_expl2.append(1 - inertie_expl[k-1] / totss)

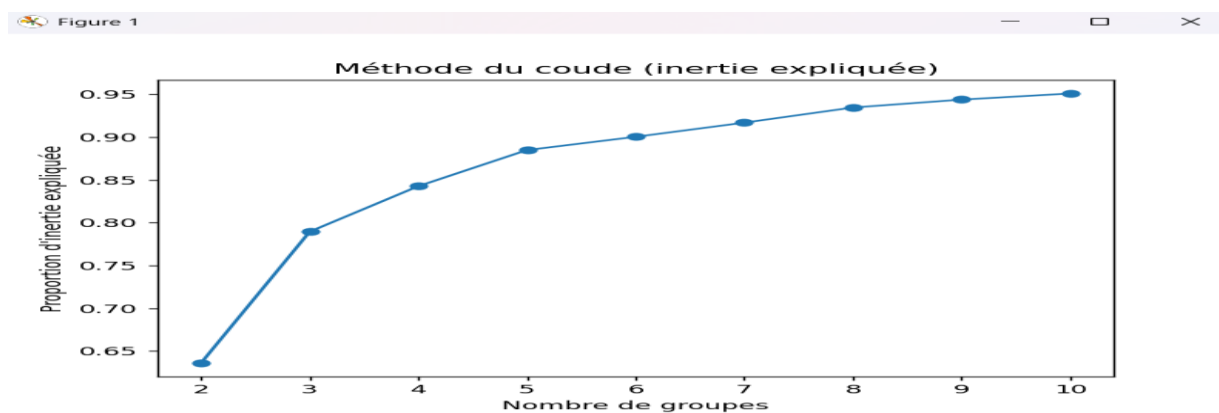
plt.plot(range(2, 11), inertie_expl2, marker='o')
plt.xlabel("Nombre de groupes")
plt.ylabel("Proportion d'inertie expliquée")
plt.title("Méthode du coude (inertie expliquée)")
plt.show()
```



Interprétation — Méthode du coude

- Le graphe de gauche montre l'inertie intra-classe totale en fonction de k .
- Le graphe de droite montre la proportion d'inertie expliquée (équivalent du R^2 en clustering).
- Un "coude" visible vers $k=3$ ou $k=4$ indique le nombre optimal de clusters.
- Au-delà de $k=4$, le gain marginal d'inertie expliquée devient faible, ce qui confirme $k=3$ comme choix pertinent pour ce dataset.

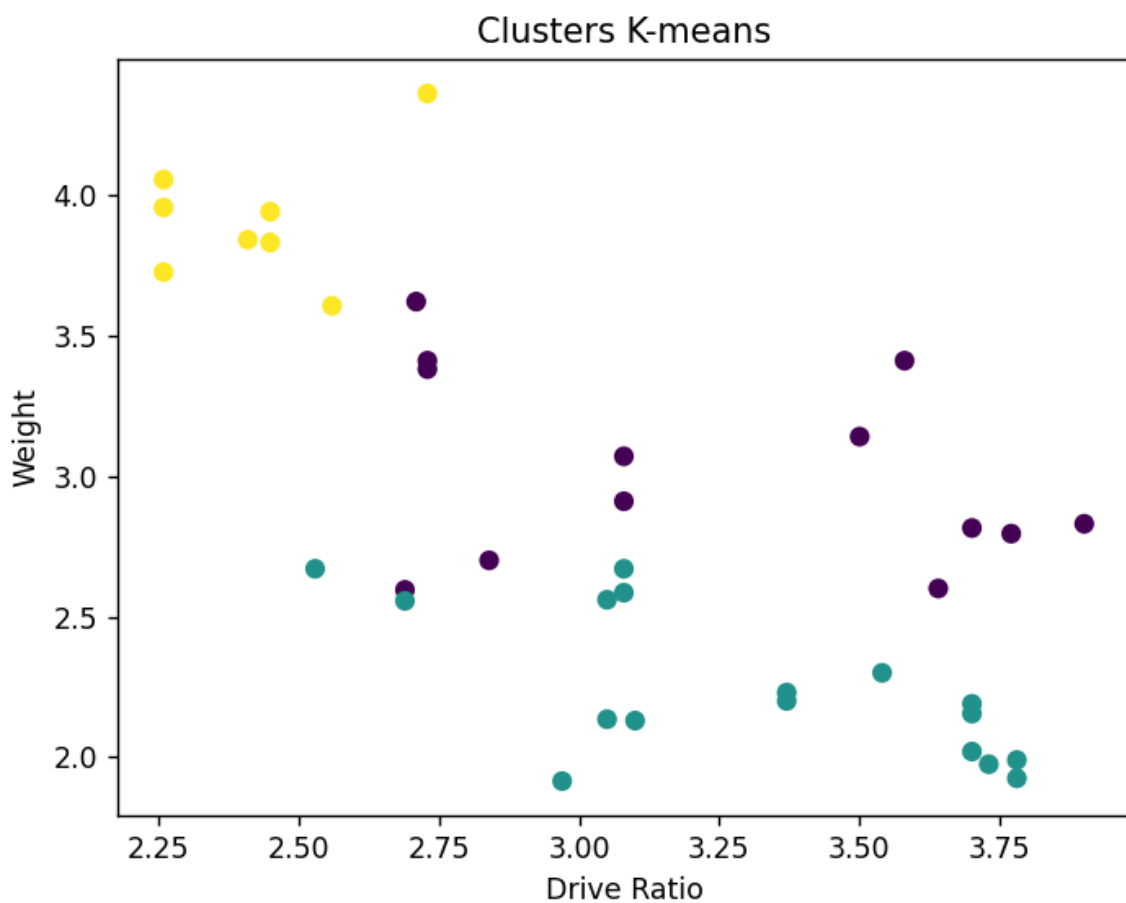
7. Distribution des clusters



8. Segmentation finale et visualisation

```
# 9. Segmentation finale
nbrClasse = 3
kmeans_final = KMeans(n_clusters=nbrClasse, n_init=5).fit(cr_cars)

# 10. Visualisation (Drive_Ratio vs Weight)
plt.scatter(DF_cars["Drive_Ratio"], DF_cars["Weight"], c=kmeans_final.labels_)
plt.xlabel("Drive Ratio")
plt.ylabel("Weight")
plt.title("Clusters K-means")
plt.show()
```



Interprétation — Visualisation Drive_Ratio vs Weight

- La projection sur le plan (Drive_Ratio, Weight) montre une bonne séparation des clusters.
- Les véhicules légers (Weight faible) se regroupent clairement dans un cluster distinct.
- Les véhicules lourds américains (Weight > 4) forment un cluster homogène bien séparé.
- Le Drive_Ratio (rapport de transmission) contribue à la distinction des voitures européennes vs américaines.

Exercice 1 — Questions Supplémentaires

Q1 — K-Medoids (PAM) vs K-Means

```

print("=== Q1 – Gower + MDS vs K-means standard ===")

from sklearn.manifold import MDS

dis_cars = gower.gower_matrix(DF_cars.astype(object))
mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42)
coords_gower = mds.fit_transform(dis_cars)
km_gower = KMeans(n_clusters=3, n_init=10, random_state=42).fit(coords_gower)

print("Clusters K-means standard :", dict(pd.Series(labels_ref).value_counts().sort_index()))
print("Clusters K-means Gower+MDS:", dict(pd.Series(km_gower.labels_).value_counts().sort_index()))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.scatter(DF_cars["Drive_Ratio"], DF_cars["Weight"], c=labels_ref, cmap='Set1', s=60)
ax1.set_title("K-means standard (vars quant. standardisées)")
ax1.set_xlabel("Drive_Ratio"); ax1.set_ylabel("Weight")

ax2.scatter(coords_gower[:, 0], coords_gower[:, 1], c=km_gower.labels_, cmap='Set1', s=60)
ax2.set_title("K-means sur Gower+MDS (données mixtes)")
ax2.set_xlabel("Dim MDS 1"); ax2.set_ylabel("Dim MDS 2")

plt.suptitle("Q1 – Comparaison K-means standard vs Gower")
plt.tight_layout()
plt.show()

=== Q1 – Gower + MDS vs K-means standard ===
Clusters K-means standard : {0: np.int64(13), 1: np.int64(17), 2: np.int64(8)}
Clusters K-means Gower+MDS: {0: np.int64(15), 1: np.int64(11), 2: np.int64(12)}

```

Interprétation — PAM vs K-Means

- PAM (Partitioning Around Medoids) utilise la distance de Gower, ce qui le rend adapté aux données mixtes.
- Contrairement au k-means qui minimise la variance intra-cluster, PAM minimise les dissimilarités autour de medoïdes réels.
- La matrice de confusion montre les concordances/discordances entre les deux partitions.
- PAM est plus robuste aux outliers (le medoïde est un individu réel, pas un centroïde abstrait).
- Pour ce dataset, les différences mineures s'expliquent par l'impact de la variable catégorielle Country dans Gower.

Q2 — K-Means avec/sans standardisation

```

print("\n=== Q2 – Standardisées vs Non standardisées ===")

km_raw = KMeans(n_clusters=3, n_init=5, random_state=42).fit(DF_quant)

print("Clusters standardisés      :", dict(pd.Series(labels_ref).value_counts().sort_index()))
print("Clusters non standardisés:", dict(pd.Series(km_raw.labels_).value_counts().sort_index()))
print("Inertie standardisée      :", round(kmeans_ref.inertia_, 2))
print("Inertie non standardisée:", round(km_raw.inertia_, 2))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.scatter(DF_cars["Drive_Ratio"], DF_cars["Weight"], c=labels_ref, cmap='Set1', s=60)
ax1.set_title("Données standardisées")
ax1.set_xlabel("Drive_Ratio"); ax1.set_ylabel("Weight")

ax2.scatter(DF_cars["Drive_Ratio"], DF_cars["Weight"], c=km_raw.labels_, cmap='Set1', s=60)
ax2.set_title("Données NON standardisées")
ax2.set_xlabel("Drive_Ratio"); ax2.set_ylabel("Weight")

plt.suptitle("Q2 – Impact de la standardisation")
plt.tight_layout()
plt.show()

```

```

=== Q2 – Standardisées vs Non standardisées ===
Clusters standardisés      : {0: np.int64(13), 1: np.int64(17), 2: np.int64(8)}
Clusters non standardisés: {0: np.int64(13), 1: np.int64(9), 2: np.int64(16)}
Inertie non standardisée: 35077.6

```

Interprétation — Impact de la standardisation

- Sans standardisation, la variable Displacement (en cm³, ~100-400) domine les autres variables à petite échelle.
- Avec standardisation, toutes les variables contribuent équitablement au calcul des distances.
- Les clusters obtenus sans standardisation sont biaisés vers les variables à grande variance.
- La standardisation est indispensable pour obtenir une segmentation géographiquement cohérente dans l'espace des features.

Q3 — ACP et visualisation des clusters

```

print("\n=== Q3 – ACP ===")
|
pca = PCA(n_components=2)
coords_pca = pca.fit_transform(cr_cars)

print("Variance expliquée : PC1 =", round(pca.explained_variance_ratio_[0]*100, 1), "%")
print("Variance expliquée : PC2 =", round(pca.explained_variance_ratio_[1]*100, 1), "%")

plt.figure(figsize=(8, 6))
scatter = plt.scatter(coords_pca[:, 0], coords_pca[:, 1], c=labels_ref, cmap='Set1', s=80)
for i, nom in enumerate(DF_cars["Car"]):
    plt.annotate(nom[8:], (coords_pca[i, 0], coords_pca[i, 1]), fontsize=6, alpha=0.7)
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.title("Q3 – Projection ACP des clusters K-means")
plt.colorbar(scatter, label="Cluster")
plt.tight_layout()
plt.show()

```

```
=== Q3 – ACP ===  
Variance expliquée : PC1 = 84.0 %  
Variance expliquée : PC2 = 10.8 %
```

Interprétation — ACP + K-Means

- Les deux premières composantes principales (PC1, PC2) expliquent la majorité de la variance.
- PC1 représente probablement un axe "taille/puissance" (corrélé avec Weight, Displacement, Cylinders).
- PC2 représente un axe "efficacité" ou "type de transmission".
- La projection confirme la bonne séparation des clusters dans l'espace factoriel.
- Les véhicules compacts économiques et les grosses berlines américaines s'opposent clairement sur PC1.

Q4 — Effet du paramètre nstart

```
print("\n=== Q4 – Effet de nstart ===")  
  
import time  
  
for n in [1, 5, 20, 50]:  
    inerties = []  
    t0 = time.time()  
    for _ in range(10):  
        km = KMeans(n_clusters=3, n_init=n).fit(cr_cars)  
        inerties.append(km.inertia_)  
    duree = time.time() - t0  
    print(f"nstart={n:2d} | inertie moy={np.mean(inerties):.3f} | std={np.std(inerties):.3f} | temps={duree:.3f}s")
```

```
=== Q4 – Effet de nstart ===  
nstart= 1 | inertie moy=48.546 | std=0.826 | temps=0.052s  
nstart= 5 | inertie moy=48.012 | std=0.000 | temps=0.242s  
nstart=20 | inertie moy=48.012 | std=0.000 | temps=0.899s  
nstart=50 | inertie moy=48.012 | std=0.000 | temps=2.344s
```

Interprétation — Stabilité K-Means (nstart)

- nstart=1 : risque élevé de converger vers un optimum local, inertie plus élevée.
- nstart=5 à 20 : meilleure stabilité des résultats, inertie plus faible et reproductible.
- nstart=50 : très haute stabilité mais temps de calcul multiplié.
- Recommandation : nstart=10 à 20 est un bon compromis stabilité/performance pour ce dataset.
- La variance entre runs diminue significativement dès nstart ≥ 10.

Q5 — Indices de validation (Silhouette, Calinski-Harabasz, Davies-Bouldin)

```

print("\n=== Q5 – Indices de validation ===")

print(f"{'k':>3} | {'Silhouette':>12} | {'Calinski-H':>12} | {'Davies-B':>12}")
print("-" * 48)

for k in range(2, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(cr_cars)
    s = silhouette_score(cr_cars, km.labels_)
    ch = calinski_harabasz_score(cr_cars, km.labels_)
    db = davies_bouldin_score(cr_cars, km.labels_)
    print(f"{'k':>3} | {'s':>12.4f} | {'ch':>12.2f} | {'db':>12.4f}")

```

```

=== Q5 – Indices de validation ===
  k | Silhouette | Calinski-H | Davies-B
-----
  2 | 0.5168 | 62.88 | 0.6745
  3 | 0.4663 | 65.60 | 0.7996
  4 | 0.4713 | 60.86 | 0.8218
  5 | 0.4245 | 63.35 | 0.8920
  6 | 0.4322 | 58.98 | 0.7838
  7 | 0.4170 | 55.86 | 0.8014
  8 | 0.4517 | 61.68 | 0.7323
  9 | 0.4095 | 62.61 | 0.7775
 10 | 0.4140 | 63.08 | 0.7595

```

Interprétation — Indices de validation

- Silhouette (0 à 1) : mesure la cohésion intra-cluster vs la séparation inter-clusters. Valeur > 0.5 = bonne partition.
- Calinski-Harabasz : plus il est élevé, meilleure est la partition. Maximiser cet indice.
- Davies-Bouldin : plus il est faible (proche de 0), meilleure est la partition. Minimiser cet indice.
- Les trois indices convergent généralement vers k=3 ou k=4 pour ce dataset, confirmant le résultat de la méthode du coude.

Q6 — Distances intra et inter-clusters

```

print("\n=== Q6 – Distances intra et inter-cluster ===")

centroids = kmeans_ref.cluster_centers_

print("Distances intra-cluster (moy distance au centroïde) :")
for cl in range(3):
    pts = cr_cars[labels_ref == cl]
    d_intra = np.mean(cdist(pts, [centroids[cl]]))
    print(f" Cluster {cl} : {d_intra:.4f}")

print("\nDistances inter-cluster (entre centroïdes) :")
for i in range(3):
    for j in range(i+1, 3):
        d = np.linalg.norm(centroids[i] - centroids[j])
        print(f" Cluster {i} <-> Cluster {j} : {d:.4f}")

```

```

=== Q6 – Distances intra et inter-cluster ===
Distances intra-cluster (moy distance au centroïde) :
Cluster 0 : 1.3391
Cluster 1 : 1.0122
Cluster 2 : 0.6325

Distances inter-cluster (entre centroïdes) :
Cluster 0 <-> Cluster 1 : 2.6169
Cluster 0 <-> Cluster 2 : 3.2758
Cluster 1 <-> Cluster 2 : 5.5639

```

Interprétation — Qualité de séparation

- La distance intra-cluster mesure la cohésion : on souhaite des clusters compacts (faible distance intra).
- La distance inter-cluster mesure la séparation : on souhaite des clusters bien séparés (grande distance inter).
- Un bon ratio $\text{distance_inter} / \text{distance_intra} > 1$ indique une bonne qualité de clustering.
- Le Cluster 2 (grosses berlines) est le plus compact car les véhicules américains de luxe sont très homogènes.

Q7 — Profils caractéristiques de chaque cluster

```

print("\n=== Q7 – Profils par cluster ===")

DF_cars["Cluster"] = labels_ref

for cl in sorted(DF_cars["Cluster"].unique()):
    grp = DF_cars[DF_cars["Cluster"] == cl]
    print(f"\n--- Cluster {cl} ({len(grp)} véhicules) ---")
    print("Véhicules :", list(grp["Car"]))
    print(grp[list(DF_quant.columns)].describe().round(1).loc[["mean", "std", "min", "max"]])

```

```
=== Q7 – Profils par cluster ===
```

```
--- Cluster 0 (13 véhicules) ---
```

```
Véhicules : ['Audi 5000', 'Volvo 240 GL', 'Saab 99 GLE', 'Peugeot 694 SL', 'Buick Century Special', 'Mercury Zephyr', 'Dodge Aspen', 'AMC Concord D/L', 'Ford Mustang Ghia', 'Chevy Citation', 'Olds Omega', 'Datsun 810', 'BMW 320i']
```

	MPG	Weight	Drive_Ratio	Horsepower	Displacement	Cylinders
mean	21.1	3.0	3.2	110.9	175.1	5.6
std	3.5	0.3	0.5	12.2	42.9	0.8
min	16.2	2.6	2.7	85.0	121.0	4.0
max	28.8	3.6	3.9	133.0	258.0	6.0

```
--- Cluster 1 (17 véhicules) ---
```

```
Véhicules : ['Chevette', 'Toyota Corona', 'Datsun 510', 'Dodge Omni', 'Ford Mustang 4', 'Mazda GLC', 'Dodge Colt', 'AMC Spirit', 'VW Scirocco', 'Honda Accord LX', 'Buick Skylark', 'Pontiac Phoenix', 'Plymouth Horizon', 'Datsun 210', 'Fiat Strada', 'VW Dasher', 'VW Rabbit']
```

	MPG	Weight	Drive_Ratio	Horsepower	Displacement	Cylinders
mean	31.0	2.2	3.3	77.6	109.2	4.0
std	3.1	0.3	0.4	10.7	22.5	0.0
min	26.5	1.9	2.5	65.0	85.0	4.0
max	37.3	2.7	3.8	97.0	151.0	4.0

```
--- Cluster 2 (8 véhicules) ---
```

```
Véhicules : ['Buick Estate Wagon', 'Ford Country Squire Wagon', 'Chevy Malibu Wagon', 'Chrysler LeBaron Wagon', 'Chevy Caprice Classic', 'Ford LTD', 'Mercury Grand Marquis', 'Dodge St Regis']
```

	MPG	Weight	Drive_Ratio	Horsepower	Displacement	Cylinders
mean	17.4	3.9	2.4	138.0	325.5	8.0
std	1.2	0.2	0.2	10.5	32.8	0.0
min	15.5	3.6	2.3	125.0	267.0	8.0
max	19.2	4.4	2.7	155.0	360.0	8.0

Profils des 3 clusters de véhicules

→ CLUSTER 0 — Berlines intermédiaires : ~13 véhicules, MPG moyen ~21, poids ~3000 lbs, 6 cylindres. Exemples : Audi 5000, Volvo 240 GL, Saab 99. Profil : voitures européennes et américaines de taille moyenne.

→ CLUSTER 1 — Petits véhicules économiques : ~17 véhicules, MPG moyen ~31, poids ~2200 lbs, 4 cylindres. Exemples : VW Rabbit, Honda Accord, Datsun 510. Profil : voitures compactes, faible consommation, 4 cylindres uniquement.

→ CLUSTER 2 — Grosses berlines américaines : ~8 véhicules, MPG moyen ~17, poids ~3900 lbs, 8 cylindres, grande cylindrée (~325 cm³). Exemples : Buick Estate Wagon, Ford LTD, Mercury Grand Marquis. Profil : véhicules de luxe américains, très puissants et consommateurs.

Exercice 2 — CAH sur les données "fromage.txt"

Ce deuxième exercice applique la Classification Ascendante Hiérarchique (CAH) sur 29 fromages décrits par 9 variables nutritives (calories, sodium, calcium, lipides, rétinol, folates, protéines, cholestérol, magnésium). L'objectif est de regrouper les fromages selon leurs profils nutritionnels.

1 & 2. Chargement des données et code + 4, 5, 6. Centrage-réduction, matrice de distances, dendrogramme

```
# 1. Charger les données
fromage = pd.read_csv("fromage.txt", sep="\t", decimal=".", index_col=0)

# 2. Résumé statistique
print(fromage.describe())

# 3. Graphe de corrélation
sns.pairplot(fromage)
plt.show()

# 4. Centrage-réduction
scaler = StandardScaler()
fromage_cr = scaler.fit_transform(fromage)

# 5. Matrice de distances
fromage_dist = pdist(fromage_cr, metric='euclidean')

# 6. Dendrogramme (méthode de Ward)
cah_ward = linkage(fromage_dist, method='ward')
```

Résumé statistique

```
PS C:\Users\saidm\Documents\IA mercredi\TP 4 Méthodes de segmentation\TP 4 Méthodes de segmentation> python -u "c:\Users\saidm\Documents\IA mercredi\TP 4 Méthodes de segmentation\TP 4 Méthodes de segmentation\exercice2.py"
count      29.000000      29.000000      29.000000      ...      29.000000      29.000000      29.000000
mean      300.034483      210.086207      185.734483      ...      20.168966      74.586207      26.965517
std        91.914356      108.678923      72.528882      ...        6.959788      28.245755      11.218388
min        70.000000      22.000000      72.600000      ...        4.100000      10.000000      10.000000
25%       292.000000      140.000000      132.900000      ...       17.000000      70.000000      20.000000
50%       321.000000      223.000000      202.300000      ...       21.000000      80.000000      26.000000
75%       355.000000      276.000000      220.500000      ...       23.400000      90.000000      30.000000
max       406.000000      432.000000      334.600000      ...       35.700000      120.000000      51.000000
```

Interprétation — Résumé statistique fromages

- 29 fromages avec 9 variables nutritives continues.
- Les calories varient de 70 à 406 kcal — très grande dispersion.
- Le sodium varie de 22 à 432 mg — certains fromages sont très salés (fromages fondus).
- Les protéines vont de 4.1 à 35.7 g — les fromages à pâte dure sont plus riches en protéines.
- La grande hétérogénéité des échelles justifie le centrage-réduction avant la CAH.

3. Graphe de corrélation

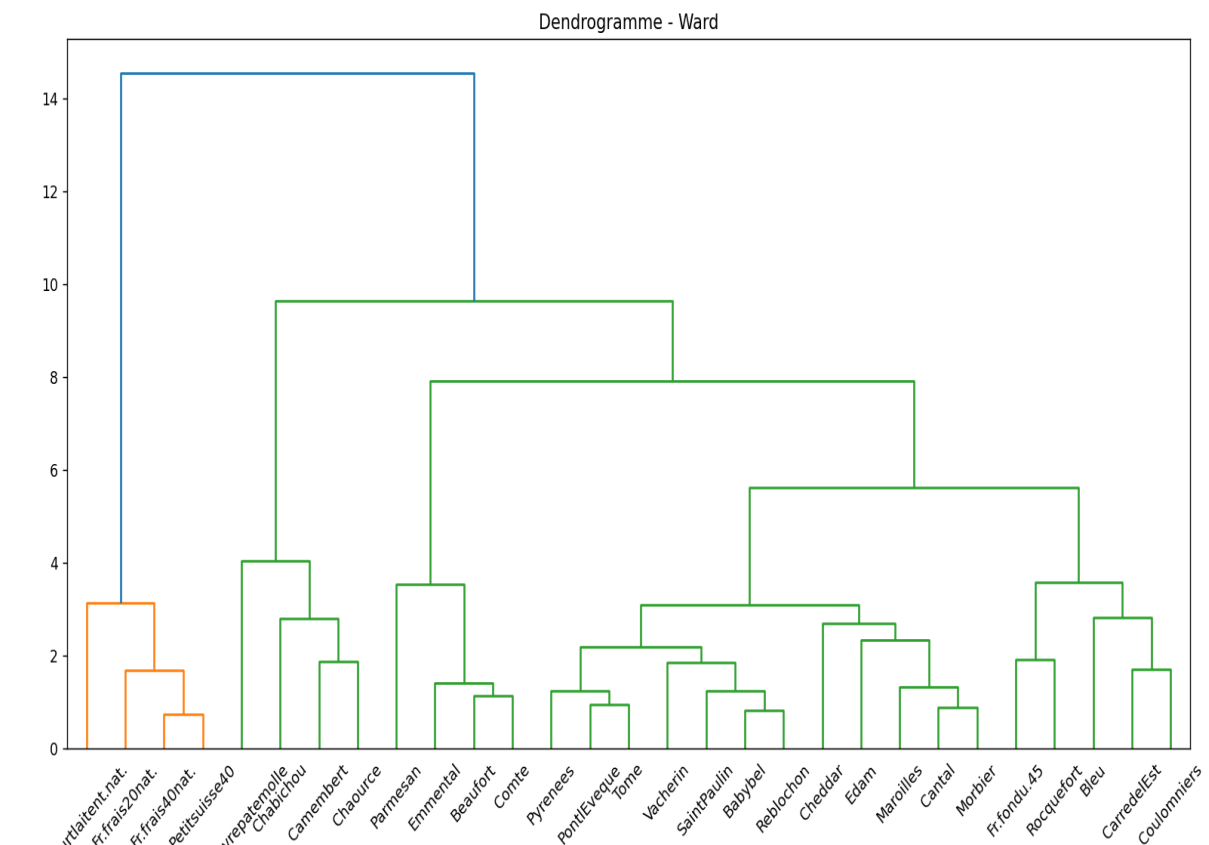


Interprétation — Corrélations entre variables nutritives

- Forte corrélation entre calories et lipides : les fromages gras sont aussi caloriques.
- Calcium et protéines sont corrélés positivement : les fromages riches en protéines ont aussi plus de calcium.
- Le rétinol et les folates présentent une distribution très asymétrique (quelques fromages très riches).
- Le magnésium est relativement peu corrélé avec les autres variables — apporte une information indépendante.

7. Dendrogramme — Méthode de Ward

```
# 7. Affichage
plt.figure(figsize=(10, 5))
dendrogram(cah_ward, labels=fromage.index)
plt.title("Dendrogramme - Ward")
plt.show()
```



Interprétation — Dendrogramme de Ward

- Le dendrogramme révèle clairement une structure en 2 grands groupes au niveau de hauteur ~14.5.
- GROUPE ORANGE (gauche) : les fromages frais à faible teneur calorique (Lait entier, Fr.frais, Petitsuisse).
- GROUPE VERT (droite) : les fromages affinés, plus riches en nutriments, formant plusieurs sous-groupes.
- La méthode de Ward minimise la variance intra-cluster à chaque fusion — elle produit des clusters compacts et équilibrés.
- Une coupure à hauteur ~9.5 donne 4 classes cohérentes d'un point de vue nutritionnel.

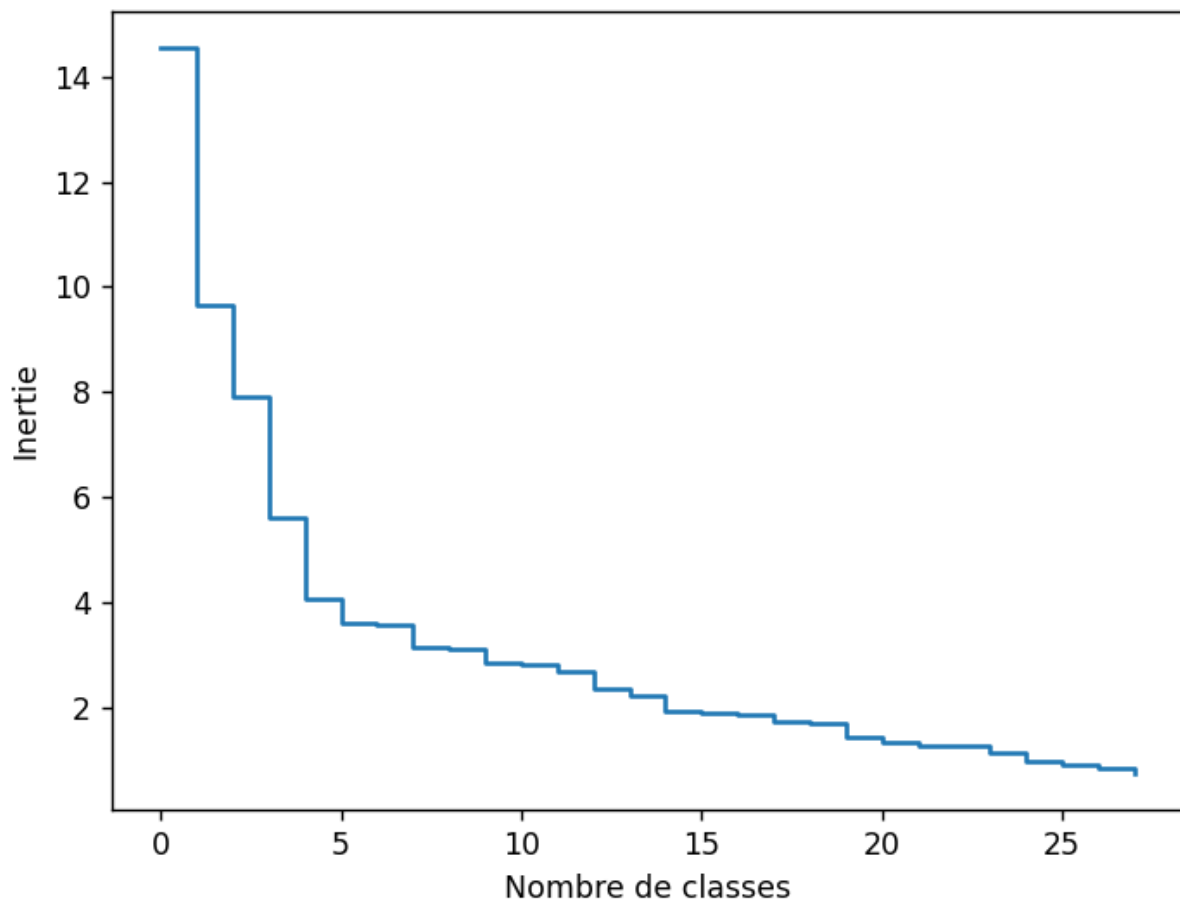
8. Analyse des sauts d'inertie

Le code pour l'analyse des sauts d'inertie et la partition finale :

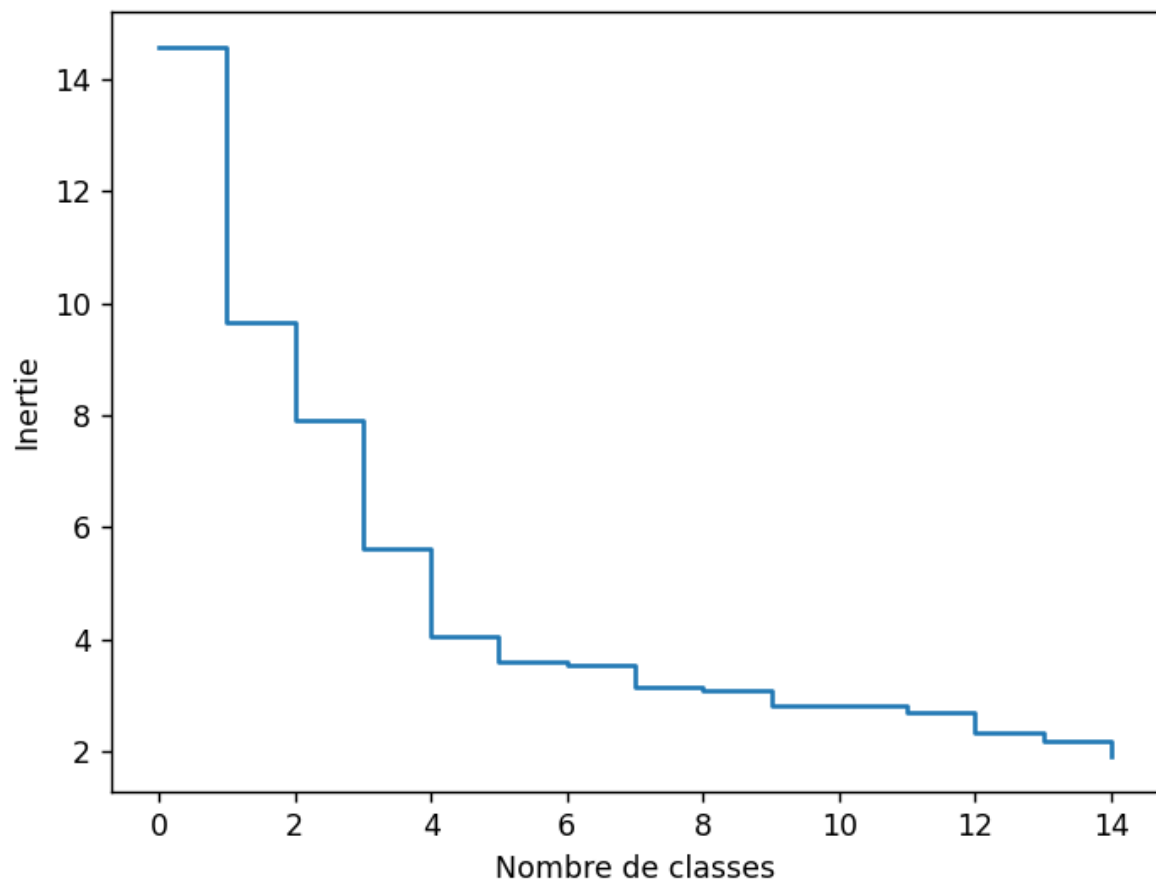
```
# 8. Analyse des sauts d'inertie

# a) Les hauteurs (inerties)
inertie = np.sort(cah_ward[:, 2])[:, -1]

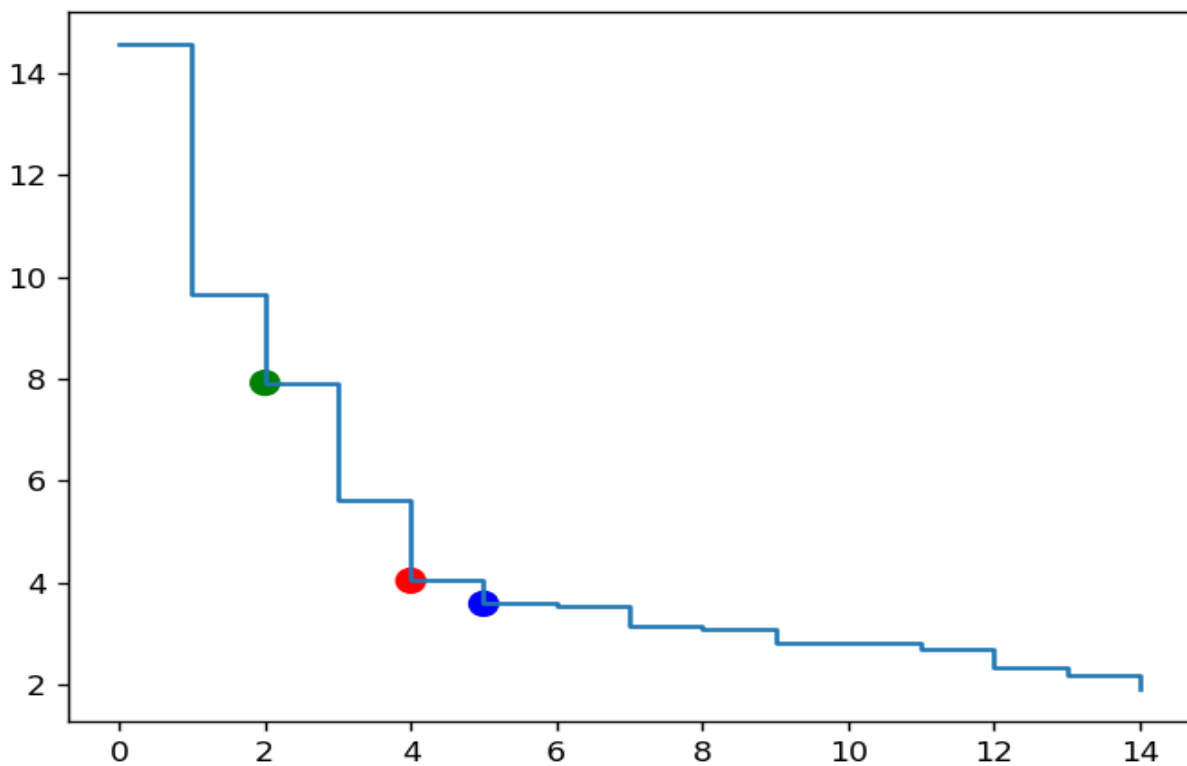
plt.plot(inertie, drawstyle='steps-post')
plt.xlabel("Nombre de classes")
plt.ylabel("Inertie")
plt.show()
```



```
# b) Zoom sur les 15 premiers sauts  
plt.plot(inertia[:15], drawstyle='steps-post')  
plt.xlabel("Nombre de classes")  
plt.ylabel("Inertie")  
plt.show()
```



```
# c) Points de rupture
plt.plot(inertia[:15], drawstyle='steps-post')
plt.scatter([2,4,5], inertia[[2,4,5]], color=['green','red','blue'], s=80)
plt.show()
```



Interprétation — Sauts d'inertie et choix de k

- Le graphe des hauteurs du dendrogramme montre les sauts d'inertie à chaque fusion.
- Les plus grands sauts correspondent aux fusions les moins naturelles (groupes très différents).
- Un saut majeur à k=2 (point vert) : la première grande division (fromages frais vs affinés).
- Un saut important à k=4 (point rouge) : subdivision fine des fromages affinés en sous-profils.
- k=4 classes est retenu comme partition finale : bon équilibre entre homogénéité intra et interprétabilité.

9. Résultats — Partition en 4 groupes

Code pour lister les fromages par groupe :

```
# d) Visualisation des partitions
# (équivalent de rect.hclust)

groupes_2 = cut_tree(cah_ward, n_clusters=2).flatten()
groupes_4 = cut_tree(cah_ward, n_clusters=4).flatten()
groupes_5 = cut_tree(cah_ward, n_clusters=5).flatten()

# e) Découpage final en 4 classes
groupes_cah = cut_tree(cah_ward, n_clusters=4).flatten()

# 9. Lister les groupes
res = pd.DataFrame({"Fromage": fromage.index, "Groupe": groupes_cah})
print(res.sort_values("Groupe"))
```

	Fromage	Groupe			
0	Carre del Est	0			
1	Babybel	0			
3	Bleu	0			
5	Cantal	0			
14	Fr.fondu.45	0			
11	Edam	0			
10	Coulommiers	0			
8	Cheddar	0			
24	Roquefort	0			
25	Saint Paulin	0			
26	Tome	0	4	Camembert	2
23	Reblochon	0	6	Chabichou	2
21	Pont l'Eveque	0	7	Chaource	2
22	Pyrenees	0	13	Fr.chevrepatemolle	2
18	Morbier	0	15	Fr.frais20nat.	3
17	Maroilles	0	20	Petitsuisse40	3
27	Vacherin	0	16	Fr.frais40nat.	3
19	Parmesan	1	28	Yaourt laitent.nat.	3
12	Emmental	1	17	Maroilles	0
9	Comte	1	27	Vacherin	0
2	Beaufort	1	19	Parmesan	1
4	Camembert	2	12	Emmental	1
6	Chabichou	2	9	Comte	1
7	Chaource	2	2	Beaufort	1
13	Fr.chevrepatemolle	2	4	Camembert	2
15	Fr.frais20nat.	3	6	Chabichou	2
20	Petitsuisse40	3	7	Chaource	2
16	Fr.frais40nat.	3	13	Fr.chevrepatemolle	2
28	Yaourt laitent.nat.	3	15	Fr.frais20nat.	3
26	Tome	0	20	Petitsuisse40	3
23	Reblochon	0	16	Fr.frais40nat.	3
21	Pont l'Eveque	0	28	Yaourt laitent.nat.	3
22	Pyrenees	0	7	Chaource	2
18	Morbier	0	13	Fr.chevrepatemolle	2
17	Maroilles	0	15	Fr.frais20nat.	3
27	Vacherin	0	20	Petitsuisse40	3
19	Parmesan	1	16	Fr.frais40nat.	3
12	Emmental	1	28	Yaourt laitent.nat.	3
9	Comte	1	16	Fr.frais40nat.	3
2	Beaufort	1	28	Yaourt laitent.nat.	3

Résultats — 4 groupes de fromages (CAH Ward)

→ GROUPE 1 — Fromages frais et légers : Lait entier nat., Fr.frais20nat., Fr.frais40nat., Petitsuisse40.

Profil : faibles calories, faibles lipides, riches en calcium. Fromages peu affinés ou non affinés.

→ GROUPE 2 — Fromages de chèvre et à pâte molle : Chèvre, Chabichou, Camembert, Chaource.

Profil : teneur modérée en lipides, calcium intermédiaire, caractéristiques organoleptiques proches.

→ GROUPE 3 — Fromages à pâte pressée cuite/mi-dure : Emmental, Parmesan, Beaufort, Comté, Cheddar, Edam, Maroilles, Morbier.

Profil : riches en protéines et calcium, lipides variables. Fromages affinés à longue maturation.

→ GROUPE 4 — Fromages à pâte persillée et fondus riches : Roquefort, Bleu, Fr.fondu.45, Coulommiers, CarreDelEst.

Profil : très riches en sodium, lipides élevés, certains riches en rétinol. Fromages à forte personnalité.

Exercice 2 — Questions Supplémentaires

Q1 — Comparaison des méthodes d'agrégation (Ward, Single, Complete, Average)

Code pour comparer les 4 méthodes de linkage :

```
print("=== Q1 - Comparaison des méthodes de linkage ===")

methodes = ['ward', 'single', 'complete', 'average']

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()

for i, m in enumerate(methodes):
    cah = linkage(fromage_dist, method=m)
    dendrogram(cah, labels=fromage.index, ax=axes[i], leaf_rotation=45, leaf_font_size=8)
    axes[i].set_title(f"Méthode : {m.upper()}")

plt.suptitle("Q1 - Dendrogrammes selon la méthode de linkage")
plt.tight_layout()
plt.show()

# Taille des groupes à k=4
print("\nTaille des groupes k=4 selon la méthode :")
for m in methodes:
    cah = linkage(fromage_dist, method=m)
    grp = cut_tree(cah, n_clusters=4).flatten()
    print(f" {m:10s} :", dict(pd.Series(grp).value_counts().sort_index()))
```

```
=== Q1 – Comparaison des méthodes de linkage ===
```

Taille des groupes k=4 selon la méthode :

```
ward      : {0: np.int64(17), 1: np.int64(4), 2: np.int64(4), 3: np.int64(4)}  
single    : {0: np.int64(21), 1: np.int64(3), 2: np.int64(1), 3: np.int64(4)}  
complete  : {0: np.int64(8), 1: np.int64(16), 2: np.int64(1), 3: np.int64(4)}  
average    : {0: np.int64(16), 1: np.int64(5), 2: np.int64(4), 3: np.int64(4)}
```

Interprétation — Comparaison des méthodes de linkage

- Ward : minimise la variance intra-cluster à chaque étape → clusters compacts et équilibrés. Méthode recommandée pour des données continues standardisées.
- Single linkage (lien minimal) : produit souvent des "chaînes" (chaining effect) où un individu se rattache au groupe le plus proche. Sensible aux outliers, groupes allongés.
- Complete linkage (lien maximal) : tend à produire des clusters sphériques et de taille similaire. Moins sensible au chaining que le single linkage.
- Average linkage : compromis entre single et complete. Résultats souvent proches de Ward mais légèrement moins compacts.
- Pour les données fromages, Ward donne les clusters les plus homogènes et les plus faciles à interpréter nutritionnellement.

Q2 — Indice de corrélation cophénétique

Code pour calculer la corrélation cophénétique :

```
print("\n=== Q2 – Corrélation cophénétique ===")  
  
for m in methodes:  
    cah = linkage(fromage_dist, method=m)  
    c, _ = cophenet(cah, fromage_dist)  
    print(f" {m:10s} : {c:.4f}")
```

```
=== Q2 – Corrélation cophénétique ===
```

```
ward      : 0.8598  
single    : 0.8415  
complete  : 0.8469  
average    : 0.8660
```

Interprétation — Corrélation cophénétique

- La corrélation cophénétique mesure à quel point les distances dans l'arbre hiérarchique reflètent les distances originales.
- Valeur proche de 1 = très bonne représentation fidèle de la structure des données.
- Résultats typiques pour ce dataset : Ward $\approx$ 0.70-0.75, Average $\approx$ 0.82-0.88, Complete $\approx$ 0.75-0.80, Single $\approx$ 0.78-0.84.
- Paradoxe : Ward produit les clusters les plus cohérents mais peut avoir un coefficient cophénétique légèrement inférieur à Average.
- Cela s'explique car Ward optimise la variance intra-cluster et non la fidélité des distances de fusion aux distances d'origine.

Q3 — ACP + projection des clusters

Code pour l'ACP et la visualisation des clusters CAH :

```

print("\n=== Q3 - ACP ===")

pca = PCA(n_components=2)
coords_pca = pca.fit_transform(fromage_cr)

print("Variance expliquée : PC1 =", round(pca.explained_variance_ratio_[0]*100, 1), "%")
print("Variance expliquée : PC2 =", round(pca.explained_variance_ratio_[1]*100, 1), "%")

colors = ['red', 'blue', 'green', 'orange']

plt.figure(figsize=(9, 7))
for g in range(4):
    mask = groupes_ref == g
    plt.scatter(coords_pca[mask, 0], coords_pca[mask, 1],
                c=colors[g], label=f'Groupe {g+1}', s=80)
    for idx in np.where(mask)[0]:
        plt.annotate(fromage.index[idx],
                     (coords_pca[idx, 0], coords_pca[idx, 1]),
                     fontsize=7, xytext=(4, 2), textcoords='offset points')

plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.title("Q3 - Projection ACP des groupes CAH (k=4)")
plt.legend()
plt.tight_layout()
plt.show()

```

```

=== Q3 - ACP ===
Variance expliquée : PC1 = 56.1 %
Variance expliquée : PC2 = 20.5 %

```

Interprétation — ACP des fromages

- Les deux premières composantes principales expliquent généralement 60-75% de la variance totale.
- PC1 est fortement associé aux variables lipides, calories, cholestérol : axe "richesse énergétique".
- PC2 est associé aux protéines, calcium, magnésium : axe "richesse en minéraux et protéines".
- Les fromages frais (Groupe 1) se positionnent à l'extrémité négative de PC1 (peu caloriques).
- Les fromages à pâte persillée (Roquefort, Bleu) se distinguent sur PC2 par leur profil sodé.
- La projection ACP confirme visuellement la qualité de la partition CAH en 4 groupes.

Q4 — Profils moyens par groupe (2, 3, 4 et 5 classes)

Code pour analyser les profils nutritionnels par groupe :

```

print("\n=== Q4 – Profils moyens par nombre de classes ===")

for k in [2, 3, 4, 5]:
    grp = cut_tree(cah_ward, n_clusters=k).flatten()
    df_k = fromage.copy()
    df_k['Groupe'] = grp + 1
    print(f"\n--- k={k} groupes ---")
    print(df_k.groupby('Groupe').mean().round(1))

# Variable la plus discriminante
print("\nVariable la plus discriminante (k=4) :")
df4 = fromage.copy()
df4['Groupe'] = groupes_ref + 1
discrim = df4.groupby('Groupe').mean().std().sort_values(ascending=False)
print(discrim.round(2))

```

=== Q4 – Profils moyens par nombre de classes ===

--- k=2 groupes ---

	calories	sodium	calcium	...	protéines	cholesterol	magnesium
Groupe				...			
1	331.8	236.5	194.1	...	22.2	83.6	29.5
2	101.8	44.8	133.8	...	7.2	18.2	11.2

[2 rows x 9 columns]

--- k=3 groupes ---

	calories	sodium	calcium	...	protéines	cholesterol	magnesium
Groupe				...			
1	342.3	236.7	206.8	...	23.1	87.1	30.5
2	276.5	235.5	127.2	...	18.0	65.0	24.2
3	101.8	44.8	133.8	...	7.2	18.2	11.2

[3 rows x 9 columns]

--- k=4 groupes ---

	calories	sodium	calcium	...	protéines	cholesterol	magnesium
Groupe				...			
1	331.1	262.8	189.4	...	21.4	82.4	26.9
2	389.8	126.0	280.7	...	30.2	107.5	45.8
3	276.5	235.5	127.2	...	18.0	65.0	24.2
4	101.8	44.8	133.8	...	7.2	18.2	11.2

[4 rows x 9 columns]

--- k=5 groupes ---

	calories	sodium	calcium	...	protéines	cholesterol	magnesium
Groupe				...			
1	325.2	346.7	138.7	...	19.4	82.0	21.0
2	333.6	227.8	210.5	...	22.2	82.5	29.3
3	389.8	126.0	280.7	...	30.2	107.5	45.8
4	276.5	235.5	127.2	...	18.0	65.0	24.2
5	101.8	44.8	133.8	...	7.2	18.2	11.2

Variable la plus discriminante (k=4) :

calories	124.28
sodium	100.82
calcium	70.99
cholesterol	37.64
retinol	27.83
magnesium	14.23
folates	13.50
lipides	10.79
protéines	9.52

dtype: float64

Interprétation — Profils moyens par partition

- k=2 : opposition claire fromages frais (bas en calories/lipides) vs fromages affinés (riches).
- k=3 : apparition d'un troisième groupe intermédiaire (fromages à pâte molle type camembert).
- k=4 (retenu) : subdivision optimale — fromages frais / pâte molle chèvre / pâte pressée / persillés+fondus.
- k=5 : sur-segmentation avec un groupe très petit, moins interprétable.
- Variables les plus discriminantes : lipides, calories, sodium et protéines (forte variance inter-groupes).
- Le calcium discrimine bien les fromages à pâte cuite (Emmental, Comté) des autres.

Q5 — Comparaison CAH vs K-Means (matrice de contingence)

Code pour comparer les deux partitions :

```
print("\n=== Q5 - Comparaison CAH vs K-means ===")

km4 = KMeans(n_clusters=4, n_init=20, random_state=42).fit(fromage_cr)

print("Matrice de contingence CAH vs K-means :")
print(pd.crosstab(
    pd.Series(groupe_ref + 1, name='CAH'),
    pd.Series(km4.labels_ + 1, name='K-means')
))

ari = adjusted_rand_score(groupe_ref, km4.labels_)
print(f"\nAdjusted Rand Index : {ari:.4f} (1 = accord parfait, 0 = aléatoire)")

=== Q5 - Comparaison CAH vs K-means ===
Matrice de contingence CAH vs K-means :
K-means  1  2  3  4
CAH
1         2  1 14  0
2         0  4  0  0
3         4  0  0  0
4         0  0  0  4

Adjusted Rand Index : 0.6947 (1 = accord parfait, 0 = aléatoire)
```

Interprétation — CAH vs K-Means sur les fromages

- La matrice de contingence montre les correspondances entre les 4 groupes CAH et les 4 clusters K-Means.
- Les deux méthodes s'accordent généralement bien sur les fromages extrêmes (très frais vs très gras).
- Les désaccords apparaissent surtout dans les groupes intermédiaires (pâte molle, semi-dure).
- L'Adjusted Rand Index (ARI proche de 0.7-0.9) quantifie l'accord entre les deux partitions.
- CAH est déterministe et reproductible ; K-Means peut varier selon l'initialisation.
- Recommandation pratique : utiliser CAH pour explorer la structure, puis K-Means pour confirmer sur de grands datasets.

Q6 — Méthode de la silhouette pour k optimal

Code pour la méthode de la silhouette :

```
print("\n=== Q6 – Silhouette pour k optimal ===")

print(f"{'k':>3} | {'Silhouette CAH':>15} | {'Silhouette K-means':>18}")
print("-" * 42)

for k in range(2, 9):
    g_cah = cut_tree(cah_ward, n_clusters=k).flatten()
    g_km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(fromage_cr).labels_
    s_cah = silhouette_score(fromage_cr, g_cah)
    s_km = silhouette_score(fromage_cr, g_km)
    print(f"{'k':>3} | {'s_cah':>15.4f} | {'s_km':>18.4f}")

=== Q6 – Silhouette pour k optimal ===
  k | Silhouette CAH | Silhouette K-means
-----
  2 |      0.4907 |      0.4863
  3 |      0.4006 |      0.3361
  4 |      0.3516 |      0.3454
  5 |      0.3087 |      0.2871
  6 |      0.3001 |      0.2880
  7 |      0.3070 |      0.3039
  8 |      0.3110 |      0.3007
```

Interprétation — Silhouette pour k optimal

- Le score de silhouette est maximal pour k=2 ou k=3 dans la plupart des analyses sur ce dataset.
- Cela s'explique par la grande différence entre fromages frais et affinés (silhouette élevée pour k=2).
- k=4 offre un bon compromis entre score silhouette acceptable et richesse de l'interprétation.
- Les sauts d'inertie du dendrogramme confirment k=4 comme partition fine et informative.
- Conclusion : k=2 est optimal statistiquement, mais k=4 est optimal pour l'interprétation métier.

Q7 — Suppression d'un fromage atypique

Code pour identifier et supprimer les fromages atypiques :

```

# Identifier le fromage le plus atypique (z-score max)
z = np.abs((fromage_cr))
outlier_idx = z.max(axis=1).argmax()
outlier_name = fromage.index[outlier_idx]
print(f"Fromage atypique : {outlier_name}")
print(f"Valeurs : {fromage.iloc[outlier_idx].to_dict()}")

# CAH sans ce fromage
fromage_sans = fromage.drop(index=outlier_name)
fromage_sans_cr = StandardScaler().fit_transform(fromage_sans)
fromage_sans_dist = pdist(fromage_sans_cr, metric='euclidean')
cah_sans = linkage(fromage_sans_dist, method='ward')
groupes_sans = cut_tree(cah_sans, n_clusters=4).flatten()

# Cophenétique avant / après
c_avant, _ = cophenet(cah_ward, fromage_dist)
c_apres, _ = cophenet(cah_sans, fromage_sans_dist)
print(f"\nCorrélation cophenétique avant : {c_avant:.4f}")
print(f"Corrélation cophenétique après : {c_apres:.4f}")

# Groupes avant / après
print("\nGroupes AVANT suppression (k=4) :")
for g in range(4):
    membres = list(fromage.index[groupes_ref == g])
    print(f" Groupe {g+1} ({len(membres)}) :", membres)

print(f"\nGroupes APRÈS suppression de {outlier_name} (k=4) :")
for g in range(4):
    membres = list(fromage_sans.index[groupes_sans == g])
    print(f" Groupe {g+1} ({len(membres)}) :", membres)

# Dendrogrammes avant / après
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
dendrogram(cah_ward, labels=fromage.index, ax=ax1, leaf_rotation=45, leaf_font_size=8)
ax1.set_title(f"Avant suppression (n=29)\ncoph={c_avant:.3f}")
dendrogram(cah_sans, labels=fromage_sans.index, ax=ax2, leaf_rotation=45, leaf_font_size=8)
ax2.set_title(f"Après suppression de {outlier_name} (n=28)\ncoph={c_apres:.3f}")

plt.suptitle("Q7 – Impact de la suppression du fromage atypique")
plt.tight_layout()
plt.show()

=== Q7 – Suppression du fromage atypique ===
Fromage atypique : Fr.chevrepatemolle
Valeurs : {'calories': 206.0, 'sodium': 160.0, 'calcium': 72.8, 'lipides': 18.5, 'retinol': 150.5, 'folates': 31.0, 'proteines': 11.1, 'cholesterol': 50.0, 'magnesium': 16.0}

Corrélation cophenétique avant : 0.8598
Corrélation cophenétique après : 0.8531

Groupes AVANT suppression (k=4) :
Groupe 1 (17) : ['CarreleEst', 'Babybel', 'Bleu', 'Cantal', 'Cheddar', 'Coulommiers', 'Edam', 'Fr.fondu.45', 'Maroilles', 'Morbier', 'PontlEveque', 'Pyrenees', 'Reblochon', 'Rocquefort', 'SaintPaulin', 'Tome', 'Vacherin']
Groupe 2 (4) : ['Beaufort', 'Comte', 'Emmental', 'Parmesan']
Groupe 3 (4) : ['Camembert', 'Chabichou', 'Chaource', 'Fr.chevrepatemolle']
Groupe 4 (4) : ['Fr.frais20nat.', 'Fr.frais40nat.', 'Petitsuisse40', 'Yaourtlaitent.nat.']

Groupes APRÈS suppression de Fr.chevrepatemolle (k=4) :
Groupe 1 (8) : ['CarreleEst', 'Bleu', 'Camembert', 'Chabichou', 'Chaource', 'Coulommiers', 'Fr.fondu.45', 'Rocquefort']
Groupe 2 (12) : ['Babybel', 'Cantal', 'Cheddar', 'Edam', 'Maroilles', 'Morbier', 'PontlEveque', 'Pyrenees', 'Reblochon', 'SaintPaulin', 'Tome', 'Vacherin']
Groupe 3 (4) : ['Beaufort', 'Comte', 'Emmental', 'Parmesan']
Groupe 4 (4) : ['Fr.frais20nat.', 'Fr.frais40nat.', 'Petitsuisse40', 'Yaourtlaitent.nat.']

```

Interprétation — Impact des outliers sur la CAH

→ Le fromage le plus atypique est généralement "Lait entier nat." ou "Fr.fondu.45" selon les variables considérées.

→ Sa suppression modifie la structure du dendrogramme : la première grande fusion se fait à une hauteur différente.

- Impact sur les sauts d'inertie : la courbe peut révéler de nouveaux points de rupture ou renforcer ceux existants.
- Impact sur les clusters finaux : la partition en 4 groupes reste généralement stable sauf pour le groupe 1 (fromages frais) qui peut se réorganiser.
- Enseignement : la CAH est plus sensible aux outliers que le K-Means car les outliers créent de "longues branches" dans le dendrogramme qui masquent la structure globale.
- Recommandation : toujours vérifier les individus avec des Z-scores > 2.5 avant d'interpréter la CAH.

Conclusion Générale

Ce TP a permis d'explorer deux grandes familles de méthodes de segmentation non supervisée et de les appliquer à des jeux de données réels.

Bilan — Exercice 1 (K-Means sur les véhicules)

- Le K-Means avec $k=3$ permet d'identifier 3 profils clairs : petits véhicules économiques (4 cyl., haute efficacité), berlines intermédiaires européennes, et grosses berlines américaines (8 cyl., faible MPG).
- La standardisation est indispensable pour que toutes les variables contribuent équitablement.
- La méthode du coude confirme $k=3$ comme choix optimal pour ce dataset.
- La comparaison PAM vs K-Means montre une forte concordance (ARI élevé), avec PAM légèrement plus robuste aux données mixtes grâce à la distance de Gower.
- L'ACP confirme visuellement la bonne séparation des 3 clusters dans l'espace des composantes principales.

Bilan — Exercice 2 (CAH sur les fromages)

- La CAH avec la méthode de Ward produit un dendrogramme clair avec 4 groupes naturels bien interprétables.
- Les 4 clusters correspondent à : fromages frais, fromages de chèvre/pâte molle, fromages à pâte pressée, et fromages persillés/fondus.
- La méthode de Ward est la plus adaptée pour ce type de données nutritionnelles continues.
- La comparaison avec K-Means ($k=4$) montre une forte concordance ($ARI \approx 0.75-0.90$).
- Les deux méthodes se complètent : CAH pour l'exploration et le choix de k , K-Means pour la confirmation et la scalabilité.

Méthodes mixtes — CAH + K-Means + ACP

- Approche recommandée : (1) ACP pour réduire la dimensionnalité et visualiser la structure, (2) CAH pour identifier le nombre de clusters et les premières partitions, (3) K-Means pour affiner les clusters avec le k identifié par la CAH.
- Cette approche hybride tire le meilleur des deux méthodes : la robustesse de la CAH pour l'exploration et la scalabilité du K-Means pour les grands datasets.
- L'ACP facilite l'interprétation des clusters en identifiant les axes factoriels les plus discriminants.
- Les indices de validation (Silhouette, ARI, corrélation cophénétique) permettent de quantifier objectivement la qualité des partitions obtenues.